

```

// =====
// PRACTICE TEST SOLUTION - TV Sales System (EF Code/DB First)
// Covers Q1.2 - Q1.7
// =====

using System;
using System.Linq;
using System.Collections.Generic;

namespace TVSalesSystem
{
    // -----
    // ENTITY CLASSES (as shown in the EF diagram, Q1.2)
    // -----
    public class Customer
    {
        public int CustID { get; set; }
        public string Name { get; set; }

        private int age;
        public int Age
        {
            get { return age; }
            set
            {
                // Q1.3: setter must keep Age in range 15-100, else default to 15
                if (value >= 15 && value <= 100)
                    age = value;
                else
                    age = 15;
            }
        }

        // Q1.3: default constructor
        public Customer() { }

        // Q1.3: overloaded constructor setting Name and Age
        public Customer(string name, int age)
        {
            Name = name;
            Age = age; // goes through the setter above, so range check still app
        }

        public void DisplayCustomerInfo()
    }
}

```

```

        {
            Console.WriteLine($"Customer ID: {CustID}\t, Name: {Name}\t, Age: {Ag
        }
    }

public class TV
{
    public int TVID { get; set; }
    public string TV_Brand { get; set; }
    public string TV_Model { get; set; }
    public decimal UnitPrice { get; set; }
    public int QOH { get; set; }

    public void DisplayTVInfo()
    {
        Console.WriteLine($"TV ID: {TVID}\t, Brand: {TV_Brand}\t, Model: {TV_
    }
}

public class TVSale
{
    public int SaleNo { get; set; }
    public int CustID { get; set; }
    public int TVID { get; set; }
    public int Qty { get; set; }
    public decimal UnitPrice { get; set; }
}

// -----
// MAIN PROGRAM CLASS
// -----
class Program
{
    // Use your actual EF DbContext here (database first).
    // Example: TVDatabaseEntities db = new TVDatabaseEntities();
    static TVDatabaseEntities db = new TVDatabaseEntities();

    static void Main(string[] args)
    {
        // ----- Q1.2: display contents of each class -----
        DisplayAllCustomers();
        DisplayAllTVs();
        DisplayAllTVSales();

        // ----- Q1.4: sorted displays -----
        DisplayCustomersSortedByName();           // ascending by Name
        DisplayTVsSortedByUnitPriceDesc();        // descending by UnitPrice
    }
}

```

```

// ----- Q1.7: execute Q1.5 and Q1.6 -----
ProcessTVSale(); // Q1.5
DisplayFinalisedSales(); // Q1.6

// Show all TVs again to confirm QOH was reduced
DisplayAllTVs();

Console.WriteLine("\nAll Done");
Console.ReadLine();
}

// =====
// Q1.2 - Void methods that display the contents of each class
// =====
static void DisplayAllCustomers()
{
    Console.WriteLine("=====List of Customers=====");
    foreach (Customer c in db.Customers)
    {
        Console.WriteLine($"Customer ID: {c.CustID}\t, Name: {c.Name}\t,
    }
}

static void DisplayAllTVs()
{
    Console.WriteLine("=====List of TVs=====");
    foreach (TV t in db.TVs)
    {
        Console.WriteLine($"TV ID: {t.TVID}\t, Brand: {t.TV_Brand}\t, Mod
    }
}

static void DisplayAllTVSales()
{
    Console.WriteLine("=====List of TV Sales=====");
    foreach (TVSale s in db.TVSales)
    {
        Console.WriteLine($"Sale No: {s.SaleNo}\t, Cust ID: {s.CustID}\t,
    }
}

// =====
// Q1.4 - Static void methods with sorting
// =====
static void DisplayCustomersSortedByName()
{

```

```

        Console.WriteLine("=====List of Customers=====");
        var sortedCustomers = db.Customers.OrderBy(c => c.Name).ToList();
        foreach (Customer c in sortedCustomers)
        {
            Console.WriteLine($"Customer ID: {c.CustID}\t, Name: {c.Name}\t,
        }
    }

static void DisplayTVsSortedByUnitPriceDesc()
{
    Console.WriteLine("=====List of TVs=====");
    var sortedTVs = db.TVs.OrderByDescending(t => t.UnitPrice).ToList();
    foreach (TV t in sortedTVs)
    {
        Console.WriteLine($"TV ID: {t.TVID}\t, Brand: {t.TV_Brand}\t, Mod
    }
}

// =====
// Q1.5 – Process the sale of TV sets
// =====
static void ProcessTVSale()
{
    while (true)
    {
        Console.Write("Enter Customer ID (or XXX to exit): ");
        string custInput = Console.ReadLine();

        if (custInput.Trim().ToUpper() == "XXX")
            break;

        int custID = Convert.ToInt32(custInput);

        Console.Write("Enter TV ID: ");
        int tvID = Convert.ToInt32(Console.ReadLine());

        // fetch the TV so we can validate quantity and update QOH
        TV selectedTV = db.TVs.FirstOrDefault(t => t.TVID == tvID);

        Console.Write("Enter Quantity: ");
        int qty = Convert.ToInt32(Console.ReadLine());

        // repeatedly prompt while requested quantity exceeds QOH
        while (qty > selectedTV.QOH)
        {
            Console.WriteLine($"Insufficient quantity. Available quantity
            Console.Write("Enter Quantity: ");

```

```

        qty = Convert.ToInt32(Console.ReadLine());
    }

    // finalise the sale - write to the TVSale table
    TVSale newSale = new TVSale
    {
        CustID = custID,
        TVID = tvID,
        Qty = qty,
        UnitPrice = selectedTV.UnitPrice
    };
    db.TVSales.Add(newSale);

    // reduce QOH
    selectedTV.QOH -= qty;

    db.SaveChanges();

    Customer cust = db.Customers.FirstOrDefault(c => c.CustID == cust
    Console.WriteLine($"Sale completed: {qty} units of {selectedTV.TV
    }
}

// =====
// Q1.6 - Display details of all finalised TV sale transactions
// =====
static void DisplayFinalisedSales()
{
    Console.WriteLine("=====List of TV Sales=====

    var sales = db.TVSales.ToList();
    int saleCounter = 1;

    foreach (TVSale s in sales)
    {
        Customer cust = db.Customers.FirstOrDefault(c => c.CustID == s.Cu
        TV tv = db.TVs.FirstOrDefault(t => t.TVID == s.TVID);
        decimal subtotal = s.Qty * s.UnitPrice;

        Console.WriteLine($"Sale No: {saleCounter}\t, Customer: {cust.Nam
            $"Quantity: {s.Qty}\t, Unit Price: {s.UnitPric
        saleCounter++;
    }
}
}
}

```

```
// -----
// NOTES FOR THE ACTUAL TEST
// -----
// 1. Replace "TVDatabaseEntities" and db.Customers / db.TVs / db.TVSales
//     with the actual DbContext class name and DbSet names EF generates
//     for your database (right-click > Add > New Item > ADO.NET Entity
//     Data Model > EF Designer/Code First from database).
// 2. The entity classes above (Customer, TV, TVSale) are usually
//     auto-generated by EF from the database (partial classes) - you
//     would extend Customer with a "partial class" to add the
//     constructors/setter from Q1.3 without editing the generated file:
//
//     public partial class Customer
//     {
//         public Customer(string name, int age) { Name = name; Age = age; }
//     }
//
//     (The Age setter with the 15-100 range check would need to replace
//     the auto-generated property, or be applied via a partial method /
//     validation in SaveChanges if the designer doesn't allow editing it directly
// 3. Column/table names (SaleNo, CustID, TVID, Qty, UnitPrice) must match
//     exactly what's in your database schema - adjust property names if needed.
```